

[CS230] Sequence Models: LSTM (Long Short-Term Memory)

Lecturer: Gemini (Integrated Editor)

□ Course Table of Contents

Chapter 1-9. Deep Learning Fundamentals & CNNs - *Completed*

Chapter 10. Sequence Models (Current Part)

- 10.1-10.3 RNN Basics & GRU - *Completed*
- **10.4 LSTM (Long Short-Term Memory)**
 - * Difference: Cell State vs Hidden State
 - * The Three Gates (Forget, Update, Output)
 - * Mathematical Equations
 - * GRU vs LSTM Comparison
- 10.5 NLP & Word Embeddings - *Upcoming*

지난 시간 복습 및 연결

지난 시간에 배운 GRU는 '기울기 소실'을 해결하기 위해 게이트를 도입한 훌륭한 모델이었습니다. 하지만 GRU는 2014년에 나온 모델이고, 그보다 훨씬 전인 1997년에 제안되어 지금까지 **시퀀스 모델의 제왕(Standard)**으로 군림하고 있는 모델이 있습니다. 바로 **LSTM**입니다. GRU가 2개의 게이트로 효율성을 추구했다면, LSTM은 3개의 게이트로 기억을 더욱 정교하게 제어합니다. 앤드류 응 교수님은 말합니다. "무엇을 쓸지 모르겠다면, 일단 LSTM부터 시작하십시오."

1 Unit Overview

□ 핵심 목표

이 단원은 시퀀스 모델의 업계 표준인 LSTM의 구조와 동작 원리를 파헤칩니다.

- **구조:** Cell State(c)와 Hidden State(a)가 분리된 이중 트랙 구조를 이해합니다.
- **게이트:** Forget, Update, Output이라는 3개의 게이트 역할을 파악합니다.
- **수식:** 과거를 잊고(Γ_f) 새로운 기억을 더하는(Γ_u) 독립적 제어 공식을 익힙니다.
- **비교:** GRU와 LSTM의 장단점을 비교하고 선택 기준을 세웁니다.

2 Essential Terminology

용어	기호	역할
Cell State	$c^{(t)}$	장기 기억 고속도로. 내부에서만 순환하며 정보를 보존함.
Hidden State	$a^{(t)}$	단기 상태 및 출력. Cell State를 가공하여 외부로 내보냄.
Forget Gate	Γ_f	과거의 기억을 삭제하는 비율 (0 1).
Update Gate	Γ_u	새로운 기억을 저장하는 비율 (0 1).
Output Gate	Γ_o	현재 상태를 다음 층으로 내보내는 비율 (0 1).

3 Core Concepts: 기억의 정교한 제어

3.1 1. The Key Difference: c and a

GRU는 기억(c)과 출력(a)이 통합되어 있었습니다. 반면 LSTM은 이를 엄격히 분리합니다.

- **Cell State ($c^{(t)}$):** 기억의 핵심입니다. 기울기가 소실되지 않도록 보호받는 경로입니다.
- **Hidden State ($a^{(t)}$):** Cell State에 $\tanh$ 를 씌우고 Output Gate를 통과시켜 만든, "지금 당장 필요한 정보"입니다.

3.2 2. The Three Gates (3개의 문지기)

LSTM은 기억을 관리하기 위해 3단계 검문을 수행합니다.

□ LSTM의 기억 관리 시스템 (직관적 비유)

- **Forget Gate (Γ_f):** [쓰레기통] "이전 기억 중 쓸모없는 건 버려라." (예: 문단이 바뀌었으니 이전 주제 삭제)
- **Update Gate (Γ_u):** [기록장] "새로운 정보 중 중요한 것만 적어라." (예: 새로운 주어 등록)
- **Output Gate (Γ_o):** [스피커] "지금 당장 필요한 정보만 말해라." (예: 다음 단어 예측에 필요한 정보만 발설)

4 Deep Dive: LSTM Equations

이 수식들이 LSTM의 작동 원리를 보여주는 지도입니다.

4.1 1. Gate Calculation

이전 상태($a^{(t-1)}$)와 현재 입력($x^{(t)}$)을 보고 게이트를 얼마나 열지 결정합니다.

$$\Gamma_f = \sigma(W_f[a^{(t-1)}, x^{(t)}] + b_f)$$

$$\Gamma_u = \sigma(W_u[a^{(t-1)}, x^{(t)}] + b_u)$$

$$\Gamma_o = \sigma(W_o[a^{(t-1)}, x^{(t)}] + b_o)$$

4.2 2. Memory Update (핵심)

□ Cell State Update (핵심 수식)

$$c^{(t)} = \underbrace{\Gamma_f \cdot c^{(t-1)}}_{\text{과거 기억 삭제}} + \underbrace{\Gamma_u \cdot \tilde{c}^{(t)}}_{\text{새 기억 추가}}$$

- **GRU와의 차이:** GRU는 Γ_u 하나로 과거와 현재의 비율을 시소처럼 조절했습니다 ($1 - \Gamma_u$).
- **LSTM:** Γ_f 와 Γ_u 가 독립적입니다. 과거를 기억하면서($\Gamma_f = 1$) 동시에 새로운 중요한 정보도 추가($\Gamma_u = 1$)할 수 있어 표현력이 더 풍부합니다.

4.3 3. Output Generation

$$a^{(t)} = \Gamma_o \cdot \tanh(c^{(t)})$$

5 Comparison: GRU vs LSTM

특징	GRU	LSTM
게이트 수	2개 (Update, Reset)	3개 (Forget, Update, Output)
복잡도	단순함 (Simpler)	복잡함 (Powerful)
데이터 양	적을 때 유리	많을 때 유리 (대용량 학습)
위상	경량화 모델	업계 표준 (Default Choice)

6 Implementation: LSTM Cell

```
1 import numpy as np
2
3 def sigmoid(x):
4     return 1 / (1 + np.exp(-x))
5
6 def lstm_cell_forward(xt, a_prev, c_prev, parameters):
7     """
8     xt: 현재 입력
9     a_prev: 이전 은닉 상태 단기()
10    c_prev: 이전 셀 상태 장기()
11    """
12    # 파라미터 로드 (Wf, Wu, Wc, Wo 등)
13    concat = np.concatenate((a_prev, xt), axis=0)
14
15    # 1. Gates 계산
16    ft = sigmoid(np.dot(parameters["Wf"], concat) + parameters["bf"]) # Forget
17    it = sigmoid(np.dot(parameters["Wu"], concat) + parameters["bu"]) # Update(Input)
18    ot = sigmoid(np.dot(parameters["Wo"], concat) + parameters["bo"]) # Output
19
20    # 2. 새로운 기억 후보 (Candidate)
21    c_tilde = np.tanh(np.dot(parameters["Wc"], concat) + parameters["bc"])
22
23    # 3. Cell State 업데이트 핵심(!)
24    # 과거를 잊을 건 잊고(ft), 새것을 더함(it)
25    c_next = ft * c_prev + it * c_tilde
```

```

26
27 # 4. Hidden State 업데이트 출력()
28 a_next = ot * np.tanh(c_next)
29
30 return a_next, c_next

```

Listing 1: LSTM Cell Forward Implementation

7 FAQ & Pitfalls

Q. Peephole Connection이란 뭔가요?

A. 기본 LSTM에서 게이트(Γ)는 $a^{(t-1)}$ 와 $x^{(t)}$ 만 봅니다. Peephole 변형은 게이트가 **Cell State**($c^{(t-1)}$)도 훑쳐보고(**Peep**) 결정을 내리게 합니다. "기억통이 얼마나 찼는지 보고 문을 열지 말지 정한다"는 개념입니다.

Q. 왜 LSTM이 기울기 소실에 강한가요?

A. $c^{(t)} = c^{(t-1)} + \dots$ 형태의 덧셈 연산 때문입니다. 역전파 시 덧셈은 기울기를 그대로($\times 1$) 전달하는 특성이 있어, 깊은 시간까지 오차가 잘 전달됩니다. (ResNet과 유사 원리)

다음 단계 (Next Step)

이제 우리는 시퀀스 데이터를 처리하는 가장 강력한 엔진(LSTM)을 만들었습니다. 이제 이 엔진에 넣을 연료(데이터)를 가공할 차례입니다.

컴퓨터는 '사과', '바나나'를 이해하지 못합니다. 숫자로 바꿔야 하는데, 단순히 1, 2로 바꾸면 단어 간 의미 관계가 사라집니다. 다음 시간에는 단어의 의미를 벡터 공간에 매핑하여 "왕 - 남자 + 여자 = 여왕"이라는 연산을 가능하게 하는 마법, [Word Embedding (Word2Vec, GloVe)]에 대해 다루겠습니다.

□ 단원 요약 (Cheat Sheet)

1. **LSTM:** 3개의 게이트로 기억을 관리하는 시퀀스 모델의 표준.
2. **Structure:** 장기 기억(c)과 단기 상태(a)를 분리하여 운영한다.
3. **Gates:** Forget(삭제), Update(추가), Output(출력).
4. **Strategy:** 일단 LSTM을 기본으로 쓰고, 경량화가 필요하면 GRU를 고려하라.